# BUSA8090 - Data and Visualisation for Business
# Assignment 1

# Urban Eats Café Chain: Analytics and Database Design Report

**Student Name: Kuan-Yin Lin**

**Student ID: 48054798**

# Part A – Database Design and Development

## 1.1 Entity Identification

| Entity Name | Entity Description |
|---|---|
| 1. OUTLET | Represents each café location |
| 2. STAFF | Represents all employees of the café chain. |
| 3. SHIFT | Specifies a work period at a specific outlet. |
| 4. CUSTOMER | Represents the café's clients |
| 5. RESERVATION | Stores information about customer bookings |
| 6. PRODUCT_CATEGORY | Show the categories of products sold in the café. |
| 7. PRODUCT | Group products into logical categories |
| 8. CUSTOMER_ORDER | Captures the details of a customer's order. |
| 9. PAYMENT | Records payment transactions against orders. |
| 10. ORDER_DETAIL | Represent what products the customer buys, price, and quantity in each order |
| 11. ROSTER | Shows when the shift is, who is rostered, and how long the staff works |

## 1.2 Identify Attributes, Primary Keys and Foreign Keys

| Entity Name | Attributes |
|---|---|
| 1. OUTLET | Outlet_ID (PK), Outlet_Name, Suburb, State, Postcode, Store_Phone_Num |
| 2. STAFF | Staff_ID (PK), Outlet_ID (FK), Staff_First_Name, Staff_Last_Name, Job_Role(Manager/Chef/Server/Barista), Staff_Phone_Num, Hire_Date |
| 3. SHIFT | Shift_ID (PK), Outlet_ID (FK), Shift_Date, Start_Time, End_Time |
| 4. CUSTOMER | Customer_ID (PK), Customer_First_Name, Customer_Last_Name, Email, Customer_Phone_Number, Member_Since |
| 5. RESERVATION | Reservation_ID (PK), Customer_ID (FK), Outlet_ID (FK), Reservation_Datetime, Number_Of_Guests, Reservation_Status(requested/confirmed/cancelled/no_show) |
| 6. PRODUCT_CATEGORY | Category_ID (PK), Category_Name (beverages, meals, desserts), Category_Description |
| 7. PRODUCT | Product_ID (PK), Category_ID (FK), Product_Name, Unit_Price, Product_Description |
| 8. CUSTOMER_ORDER | Order_ID (PK), Customer_Id (FK), Outlet_ID (FK), Order_Datetime, Order_Type(dine in, takeaway, and |

| | |
|---|---|
| | delivery), Order_Status(open/paid/void/refunded), Subtotal, Tax , Discount_Total , Grand_Total, Delivery_Address |
| 9. PAYMENT | Payment_ID(PK),Order_ID (FK), Payment_Datetime, Amount, Payment_Method(cash/card/online/giftcard), Payment_Status(approved/failed/refunded) . |
| 10. ROSTER | Shift_ID(PK), Staff_ID(PK), Outlet_ID, Scheduled_Start_Time, Scheduled_End_Time, Actual_Start_Time, Actual_End_Time, Role_On_Shift (Manager/Chef/Server/Barista), Hours_Worked |
| 11. ORDER_DETAIL | Order_ID(PK), Line_No(PK), Product_ID , Unit_Price_At_Order, Quantity, Line_Discount , Tax_Rate_At_Order, Line_Total |

## 1.3 Relationship Definition and Cardinality Analysis

OUTLET --- 1 --- employs --- M --- STAFF

One outlet can employ many staff; each staff member belongs to exactly one home outlet.

OUTLET --- 1 --- schedules --- M --- SHIFT

One outlet schedules many shifts; each shift occurs at exactly one outlet

OUTLET --- 1 --- receives --- M --- RESERVATION

An outlet receives many reservations; each reservation is for exactly one outlet.

OUTLET --- 1 --- processes --- M --- ORDER

Each order is processed by exactly one outlet; an outlet can process many orders.

CUSTOMER --- 1 --- makes --- M --- RESERVATION

A customer may make many reservations; each reservation is made by exactly one customer.

CUSTOMER --- 1 --- places --- M --- ORDER (CUSTOMER_ORDER)

A customer places many orders; each order can only be placed by one customer.

PRODUCT_CATEGORY --- 1 --- categorises --- M --- PRODUCT

A product belongs to exactly one category; a category can classify many products.

ORDER(CUSTOMER_ORDER) --- 1 --- pays_for --- M --- PAYMENT

An order can have zero or many payments (open/unpaid); each payment is for exactly one order.

ORDER(CUSTOMER_ORDER) -- "M" --- contains --- "M" -- PRODUCT

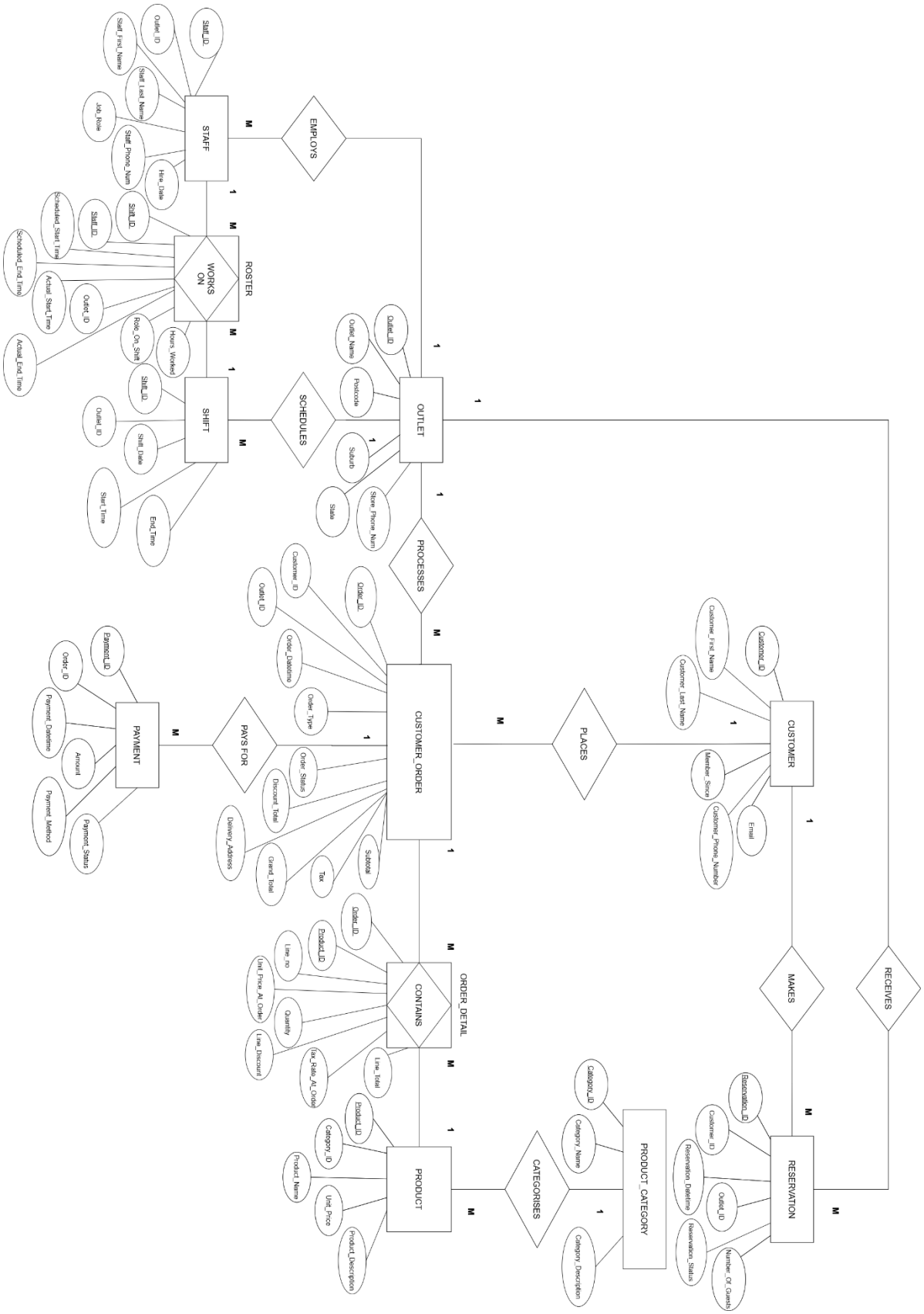
An order contains multiple products, and a product can be in multiple orders.

STAFF --- M --- works_on --- M --- SHIFT

A staff member works multiple shifts, and a shift has multiple staff.

## 1.4 ERD diagram

<<insert your ER Diagram on this page, change to landscape orientation if required>>



## 1.5 Table implementation

**Entity 1: OUTLET**

```sql
/* 1) OUTLET */
CREATE TABLE OUTLET (
  Outlet_ID          INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  Outlet_Name        VARCHAR(100) NOT NULL,
  Suburb             VARCHAR(80)  NOT NULL,
  State              VARCHAR(30)  NOT NULL,
  Postcode           CHAR(4)      NOT NULL,
  Store_Phone_Num    VARCHAR(20)  NULL,
  UNIQUE KEY uq_outlet_name (Outlet_Name)
);
```

**Entity 2 : STAFF**

```sql
/* 2) STAFF */
CREATE TABLE STAFF (
  Staff_ID           INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  Outlet_ID          INT UNSIGNED NOT NULL,
  Staff_First_Name   VARCHAR(60)  NOT NULL,
  Staff_Last_Name    VARCHAR(60)  NOT NULL,
  Job_Role           VARCHAR(20)  NOT NULL,
  Staff_Phone_Num    VARCHAR(20)  NULL,
  Hire_Date          DATE         NOT NULL,
  CONSTRAINT ck_staff_role CHECK (Job_Role IN ('Manager','Chef','Server','Barista')),
  CONSTRAINT fk_staff_outlet
    FOREIGN KEY (Outlet_ID) REFERENCES OUTLET(Outlet_ID)
      ON UPDATE CASCADE ON DELETE RESTRICT,
  -- helps ROSTER outlet-matching rule
  UNIQUE KEY uq_staff_outlet_staff (Outlet_ID, Staff_ID),
  KEY ix_staff_outlet (Outlet_ID),
  KEY ix_staff_name (Staff_Last_Name, Staff_First_Name)
);
```

### Entity 3: SHIFT

```sql
/* 3) SHIFT */
CREATE TABLE SHIFT (
  Shift_ID           INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  Outlet_ID          INT UNSIGNED NOT NULL,
  Shift_Date         DATE NOT NULL,
  Start_Time         TIME NOT NULL,
  End_Time           TIME NOT NULL,
  CONSTRAINT ck_shift_time CHECK (Start_Time < End_Time),
  CONSTRAINT fk_shift_outlet
    FOREIGN KEY (Outlet_ID) REFERENCES OUTLET(Outlet_ID)
      ON UPDATE CASCADE ON DELETE RESTRICT,
  -- allow composite FK from ROSTER to assert same-outlet pairing
  UNIQUE KEY uq_shift_outlet_shift (Outlet_ID, Shift_ID),
  -- prevent accidental duplicate shift windows at an outlet
  UNIQUE KEY uq_shift_window (Outlet_ID, Shift_Date, Start_Time, End_Time),
  KEY ix_shift_date (Shift_Date)
);

```

### Entity 4: CUSTOMER

```sql
/* 4) CUSTOMER */
CREATE TABLE CUSTOMER (
  Customer_ID            INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  Customer_First_Name    VARCHAR(60) NOT NULL,
  Customer_Last_Name     VARCHAR(60) NOT NULL,
  Email                  VARCHAR(120) NULL,
  Customer_Phone_Number  VARCHAR(20)  NULL,
  Member_Since           DATE         NOT NULL,
  UNIQUE KEY uq_customer_email (Email),
  KEY ix_customer_name (Customer_Last_Name, Customer_First_Name)
);

```

### Entity 5: RESERVATION

```sql
/* 5) RESERVATION */
CREATE TABLE RESERVATION (
  Reservation_ID       INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  Customer_ID          INT UNSIGNED NOT NULL,
  Outlet_ID            INT UNSIGNED NOT NULL,
  Reservation_Datetime DATETIME     NOT NULL,
  Number_Of_Guests     INT UNSIGNED NOT NULL,
  Reservation_Status   VARCHAR(12)  NOT NULL,
  CONSTRAINT ck_res_status CHECK (Reservation_Status IN ('requested','confirmed','cancelled','no_show')),
  CONSTRAINT fk_res_customer
    FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID)
      ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_res_outlet
    FOREIGN KEY (Outlet_ID) REFERENCES OUTLET(Outlet_ID)
      ON UPDATE CASCADE ON DELETE RESTRICT,
  KEY ix_res_outlet_dt (Outlet_ID, Reservation_Datetime),
  KEY ix_res_customer_dt (Customer_ID, Reservation_Datetime)
);
```

### Entity 6: PRODUCT_CATEGORY

```
80      /* 6) PRODUCT_CATEGORY */
81    CREATE TABLE PRODUCT_CATEGORY (
82      Category_ID        INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
83      Category_Name      VARCHAR(40)  NOT NULL,
84      Category_Description VARCHAR(200) NULL,
85      UNIQUE KEY uq_category_name (Category_Name)
86    );
```

### Entity 7: PRODUCT

```
88      /* 7) PRODUCT */
89    CREATE TABLE PRODUCT (
90      Product_ID         INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
91      Category_ID        INT UNSIGNED NOT NULL,
92      Product_Name       VARCHAR(120) NOT NULL,
93      Unit_Price         DECIMAL(10,2) NOT NULL,
94      Product_Description VARCHAR(300) NULL,
95      CONSTRAINT fk_product_category
96        FOREIGN KEY (Category_ID) REFERENCES PRODUCT_CATEGORY(Category_ID)
97          ON UPDATE CASCADE ON DELETE RESTRICT,
98      UNIQUE KEY uq_product_name (Product_Name),
99      KEY ix_product_category (Category_ID)
100   );
```

### Entity 8: CUSTOMER ORDER

```
102     /* 8) CUSTOMER_ORDER */
103   CREATE TABLE CUSTOMER_ORDER (
104     Order_ID           INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
105     Customer_ID        INT UNSIGNED NOT NULL,
106     Outlet_ID          INT UNSIGNED NOT NULL,
107     Order_Datetime     DATETIME     NOT NULL,
108     Order_Type         VARCHAR(10)  NOT NULL,     -- dine in / takeaway / delivery
109     Order_Status       VARCHAR(10)  NOT NULL,     -- open / paid / void / refunded
110     Subtotal           DECIMAL(10,2) NOT NULL DEFAULT 0.00,
111     Tax                DECIMAL(10,2) NOT NULL DEFAULT 0.00,
112     Discount_Total     DECIMAL(10,2) NOT NULL DEFAULT 0.00,
113     Grand_Total        DECIMAL(10,2) NOT NULL DEFAULT 0.00,
114     Delivery_Address   VARCHAR(200)  NULL,
115     CONSTRAINT ck_order_type CHECK (Order_Type IN ('dine in','takeaway','delivery')),
116     CONSTRAINT ck_order_status CHECK (Order_Status IN ('open','paid','void','refunded')),
117     CONSTRAINT ck_delivery_address CHECK (
118       Order_Type <> 'delivery' OR Delivery_Address IS NOT NULL
119     ),
```

```
    CONSTRAINT fk_order_customer
      FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_order_outlet
      FOREIGN KEY (Outlet_ID) REFERENCES OUTLET(Outlet_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    KEY ix_order_outlet_dt (Outlet_ID, Order_Datetime),
    KEY ix_order_customer_dt (Customer_ID, Order_Datetime),
    KEY ix_order_status (Order_Status)
);

```

### Entity 9: ORDER_DETAIL

```
/* 9) ORDER_DETAIL  (PK uses (Order_ID, Line_No)) */
CREATE TABLE ORDER_DETAIL (
    Order_ID             INT UNSIGNED NOT NULL,
    Line_No              INT UNSIGNED NOT NULL,
    Product_ID           INT UNSIGNED NOT NULL,
    Unit_Price_At_Order  DECIMAL(10,2) NOT NULL,
    Quantity             INT UNSIGNED NOT NULL,
    Line_Discount        DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    Tax_Rate_At_Order    DECIMAL(5,2)  NOT NULL,  -- e.g., 10.00 for 10%
    Line_Total           DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (Order_ID, Line_No),
    CONSTRAINT fk_od_order
      FOREIGN KEY (Order_ID) REFERENCES CUSTOMER_ORDER(Order_ID)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_od_product
      FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    -- enable "one product per order" rule if you want Cape-Codd-style uniqueness:
    -- UNIQUE KEY uq_od_order_product (Order_ID, Product_ID),
    KEY ix_od_product (Product_ID)
);

```

## Entity 10: PAYMENT

```sql
/* 10) PAYMENT */
CREATE TABLE PAYMENT (
  Payment_ID        INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  Order_ID          INT UNSIGNED NOT NULL,
  Payment_Datetime  DATETIME     NOT NULL,
  Amount            DECIMAL(10,2) NOT NULL,
  Payment_Method    VARCHAR(10)  NOT NULL,     -- cash/card/online/giftcard
  Payment_Status    VARCHAR(10)  NOT NULL,     -- approved/failed/refunded
  CONSTRAINT ck_payment_method CHECK (Payment_Method IN ('cash','card','online','giftcard')),
  CONSTRAINT ck_payment_status CHECK (Payment_Status IN ('approved','failed','refunded')),
  CONSTRAINT ck_payment_amount CHECK (Amount > 0),
  CONSTRAINT fk_payment_order
    FOREIGN KEY (Order_ID) REFERENCES CUSTOMER_ORDER(Order_ID)
      ON UPDATE CASCADE ON DELETE RESTRICT,
  KEY ix_payment_order (Order_ID),
  KEY ix_payment_dt (Payment_Datetime)
);

```

## Entity 11: ROSTER

```sql
/* 11) ROSTER  (junction for Staff-Shift with outlet integrity) */
CREATE TABLE ROSTER (
  Shift_ID             INT UNSIGNED NOT NULL,
  Staff_ID             INT UNSIGNED NOT NULL,
  Outlet_ID            INT UNSIGNED NOT NULL,   -- enforces same-outlet pairing
  Scheduled_Start_Time DATETIME     NOT NULL,
  Scheduled_End_Time   DATETIME     NOT NULL,
  Actual_Start_Time    DATETIME     NULL,
  Actual_End_Time      DATETIME     NULL,
  Role_On_Shift        VARCHAR(20)  NOT NULL,
  Hours_Worked         DECIMAL(6,2) AS (
    CASE
      WHEN Actual_Start_Time IS NULL OR Actual_End_Time IS NULL
        THEN NULL
      ELSE ROUND(TIMESTAMPDIFF(MINUTE, Actual_Start_Time, Actual_End_Time)/60, 2)
    END
  ) STORED,
  CONSTRAINT ck_roster_sched CHECK (Scheduled_Start_Time < Scheduled_End_Time),
  CONSTRAINT ck_roster_actual CHECK (
    Actual_Start_Time IS NULL OR Actual_End_Time IS NULL OR Actual_Start_Time < Actual_End_Time
  ),

  CONSTRAINT ck_roster_role CHECK (Role_On_Shift IN ('Manager','Chef','Server','Barista')),
  PRIMARY KEY (Shift_ID, Staff_ID),
  CONSTRAINT fk_roster_shift
    FOREIGN KEY (Outlet_ID, Shift_ID) REFERENCES SHIFT(Outlet_ID, Shift_ID)
      ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_roster_staff
    FOREIGN KEY (Outlet_ID, Staff_ID) REFERENCES STAFF(Outlet_ID, Staff_ID)
      ON UPDATE CASCADE ON DELETE RESTRICT,
  KEY ix_roster_staff (Staff_ID),
  KEY ix_roster_shift (Shift_ID)
);
```

## 1.6 Dummy Data insertion

### Entity 1: OUTLET

```
209     -- ==========================
210     -- OUTLET (exactly 3)
211     -- ==========================
212     INSERT INTO OUTLET (Outlet_ID, Outlet_Name, Suburb, State, Postcode, Store_Phone_Num) VALUES
213       (1, 'Urban Eats CBD',        'Sydney CBD', 'NSW', '2000', '0291000001'),
214       (2, 'Urban Eats Newtown',    'Newtown',    'NSW', '2042', '0291000002'),
215       (3, 'Urban Eats Parramatta', 'Parramatta', 'NSW', '2150', '0291000003');
```

### Entity 2 : STAFF

```
217     -- ========================
218     -- STAFF (4 core roles per outlet; no cross-outlet staff)
219     -- ========================
220     INSERT INTO STAFF (Staff_ID, Outlet_ID, Staff_First_Name, Staff_Last_Name, Job_Role, Staff_Phone_Num, Hire_Date) VALUES
221       -- Outlet 1
222       (1, 1, 'Ava',   'Nguyen',  'Manager', '0400000001', '2023-01-10'),
223       (2, 1, 'Liam',  'Patel',   'Chef',    '0400000002', '2023-03-04'),
224       (3, 1, 'Mia',   'Zhou',    'Server',  '0400000003', '2024-02-12'),
225       (4, 1, 'Noah',  'Price',   'Barista', '0400000004', '2024-06-01'),
226       -- Outlet 2
227       (5, 2, 'Isla',  'Brown',   'Manager', '0400000005', '2022-11-20'),
228       (6, 2, 'Ethan', 'Taylor',  'Chef',    '0400000006', '2023-08-09'),
229       (7, 2, 'Grace', 'Wilson',  'Server',  '0400000007', '2024-06-22'),
230       (8, 2, 'Jack',  'Martin',  'Barista', '0400000008', '2025-01-05'),
231       -- Outlet 3
232       (9,  3, 'Emily','Clark',   'Manager', '0400000009', '2022-07-15'),
233       (10, 3, 'Leo',  'Harris',  'Chef',    '0400000010', '2023-10-31'),
234       (11, 3, 'Sophie','Cooper', 'Server',  '0400000011', '2025-02-12'),
235       (12, 3, 'Lucas','King',    'Barista', '0400000012', '2024-09-30');
```

### Entity 3: SHIFT

```
237     -- ========================
238     -- SHIFT (2 shifts/day/outlet across 3 days = 18 shifts)
239     -- ========================

240     INSERT INTO SHIFT (Shift_ID, Outlet_ID, Shift_Date, Start_Time, End_Time) VALUES
241       -- 2025-09-15
242       (1, 1, '2025-09-15', '08:00:00', '14:00:00'),
243       (2, 1, '2025-09-15', '14:00:00', '22:00:00'),
244       (3, 2, '2025-09-15', '08:00:00', '14:00:00'),
245       (4, 2, '2025-09-15', '14:00:00', '22:00:00'),
246       (5, 3, '2025-09-15', '08:00:00', '14:00:00'),
247       (6, 3, '2025-09-15', '14:00:00', '22:00:00'),
248       -- 2025-09-16
249       (7, 1, '2025-09-16', '08:00:00', '14:00:00'),
250       (8, 1, '2025-09-16', '14:00:00', '22:00:00'),
251       (9, 2, '2025-09-16', '08:00:00', '14:00:00'),
252       (10,2, '2025-09-16', '14:00:00', '22:00:00'),
253       (11,3, '2025-09-16', '08:00:00', '14:00:00'),
254       (12,3, '2025-09-16', '14:00:00', '22:00:00'),
255       -- 2025-09-17
256       (13,1, '2025-09-17', '08:00:00', '14:00:00'),
257       (14,1, '2025-09-17', '14:00:00', '22:00:00'),
258       (15,2, '2025-09-17', '08:00:00', '14:00:00'),
259       (16,2, '2025-09-17', '14:00:00', '22:00:00'),
260       (17,3, '2025-09-17', '08:00:00', '14:00:00'),
261       (18,3, '2025-09-17', '14:00:00', '22:00:00');
```

## Entity 4: CUSTOMER

```
-- ========================
-- Customer
-- ========================
INSERT INTO CUSTOMER (Customer_ID, Customer_First_Name, Customer_Last_Name, Email, Customer_Phone_Number, Member_Since) VALUES
  (1, 'Olivia', 'Green', 'olivia.green@example.com', '0411000001', '2023-06-01'),
  (2, 'William', 'Lee', 'william.lee@example.com', '0411000002', '2023-08-15'),
  (3, 'Amelia', 'Davis', 'amelia.davis@example.com', '0411000003', '2024-01-20'),
  (4, 'James', 'Moore', 'james.moore@example.com', '0411000004', '2024-03-05'),
  (5, 'Charlotte', 'Young', 'charlotte.young@example.com', '0411000005', '2022-12-11'),
  (6, 'Benjamin', 'Scott', 'benjamin.scott@example.com', '0411000006', '2025-02-02'),
  (7, 'Harper', 'Adams', 'harper.adams@example.com', '0411000007', '2024-09-09'),
  (8, 'Elijah', 'Nelson', 'elijah.nelson@example.com', '0411000008', '2023-10-25'),
  (9, 'Ava', 'Carter', 'ava.carter@example.com', '0411000009', '2023-05-14'),
  (10, 'Lucas', 'Mitchell', 'lucas.mitchell@example.com', '0411000010', '2025-05-01'),
  (11, 'Sofia', 'Perez', 'sofia.perez@example.com', '0411000011', '2024-12-30'),
  (12, 'Mason', 'Roberts', 'mason.roberts@example.com', '0411000012', '2024-04-17'),
  (13, 'Ella', 'Turner', 'ella.turner@example.com', '0411000013', '2025-06-23'),
  (14, 'Henry', 'Phillips', 'henry.phillips@example.com', '0411000014', '2024-07-07'),
  (15, 'Lily', 'Campbell', 'lily.campbell@example.com', '0411000015', '2022-09-01');
```

## Entity 5: RESERVATION

```
-- ========================
-- RESERVATIONS
-- ========================
INSERT INTO RESERVATION (Reservation_ID, Customer_ID, Outlet_ID, Reservation_Datetime, Number_Of_Guests, Reservation_Status) VALUES
  (1, 1, 1, '2025-09-18 12:30:00', 2, 'requested'),
  (2, 2, 1, '2025-09-18 19:00:00', 4, 'confirmed'),
  (3, 3, 2, '2025-09-19 08:30:00', 1, 'confirmed'),
  (4, 4, 2, '2025-09-19 13:00:00', 3, 'cancelled'),
  (5, 5, 3, '2025-09-20 18:30:00', 5, 'confirmed'),
  (6, 6, 3, '2025-09-20 20:00:00', 2, 'requested'),
  (7, 7, 1, '2025-09-21 10:00:00', 2, 'confirmed'),
  (8, 8, 1, '2025-09-21 11:30:00', 6, 'requested'),
  (9, 9, 2, '2025-09-22 09:00:00', 1, 'no_show'),
  (10,10,3, '2025-09-22 12:30:00', 2, 'confirmed'),
  (11,11,2, '2025-09-23 18:00:00', 4, 'cancelled'),
  (12,12,3, '2025-09-24 07:30:00', 2, 'confirmed');
```

## Entity 6: PRODUCT_CATEGORY

```
-- ========================
-- PRODUCT_CATEGORY (only three)
-- ========================
INSERT INTO PRODUCT_CATEGORY (Category_ID, Category_Name, Category_Description) VALUES
  (1, 'Beverages', 'Hot and cold drinks'),
  (2, 'Meals',     'Breakfast and mains'),
  (3, 'Desserts',  'Sweets and cakes');

```

## Entity 7: PRODUCT

```sql
-- ========================
-- PRODUCT (15 items across the three categories)
-- ========================
INSERT INTO PRODUCT (Product_ID, Category_ID, Product_Name, Unit_Price, Product_Description) VALUES
  -- Beverages
  (1, 1, 'Flat White',     4.50, 'Espresso with microfoam'),
  (2, 1, 'Cappuccino',     4.50, 'Espresso with milk foam'),
  (3, 1, 'Iced Latte',     5.50, 'Cold milk over espresso'),
  (4, 1, 'Matcha Latte',   5.00, 'Green tea latte'),
  (5, 1, 'Long Black',     3.80, 'Americano style'),
  -- Meals
  (6, 2, 'Smashed Avo Toast',   14.00, 'Avocado, feta, sourdough'),
  (7, 2, 'Big Breakfast',       19.00, 'Eggs, bacon, sausage, toast'),
  (8, 2, 'Beef Burger',         18.00, 'Beef patty, cheese, fries'),
  (9, 2, 'Grilled Chicken Salad',16.50, 'Chicken, greens, dressing'),
  (10,2, 'Veggie Wrap',         12.50, 'Roasted veggies, hummus'),
  -- Desserts
  (11,3, 'Chocolate Brownie',   6.50, 'Rich chocolate brownie'),
  (12,3, 'Cheesecake Slice',    7.00, 'Classic NY cheesecake'),
  (13,3, 'Lemon Tart',          6.00, 'Tangy lemon tart'),
  (14,3, 'Ice Cream Sundae',    5.50, 'Vanilla sundae'),
  (15,3, 'Almond Croissant',    5.50, 'Buttery almond pastry');
```

## Entity 8: CUSTOMER ORDER

```sql
-- ========================
-- CUSTOMER_ORDERS (dine in / takeaway / delivery; statuses per business)
-- ========================
INSERT INTO CUSTOMER_ORDER
 (Order_ID, Customer_ID, Outlet_ID, Order_Datetime, Order_Type, Order_Status,
  Subtotal, Tax, Discount_Total, Grand_Total, Delivery_Address)
VALUES
 -- Fri (breakfast/lunch/dinner)
 (16,  1, 1,'2025-09-19 08:20:00','dine in','paid',      14.50, 1.45, 0.00, 15.95, NULL),
 (17,  2, 1,'2025-09-19 12:15:00','takeaway','paid',     22.50, 2.25, 0.00, 24.75, NULL),
 (18,  3, 2,'2025-09-19 19:10:00','delivery','paid',     48.00, 4.80, 0.00, 52.80, '12 King St, Newtown NSW'),

 -- Sat (morning/lunch/dinner)
 (19,  4, 3,'2025-09-20 09:05:00','dine in','paid',      23.30, 2.33, 0.00, 25.63, NULL),
 (20,  5, 3,'2025-09-20 12:45:00','takeaway','paid',     28.00, 2.80, 0.00, 30.80, NULL),
 (21,  6, 2,'2025-09-20 19:40:00','delivery','paid',     47.00, 4.70, 1.00, 51.70, '25 Enmore Rd, Newtown NSW'),

 -- Sun (morning/lunch/afternoon/dinner)
 (22,  7, 1,'2025-09-21 08:15:00','dine in','paid',      15.00, 1.50, 0.00, 16.50, NULL),
 (23,  8, 1,'2025-09-21 13:10:00','dine in','paid',      29.80, 2.98, 0.00, 32.78, NULL),
 (24,  9, 2,'2025-09-21 15:30:00','takeaway','paid',     17.00, 1.70, 0.00, 18.70, NULL),
 (25, 10, 3,'2025-09-21 18:50:00','delivery','paid',     45.00, 4.50, 0.00, 49.50, '90 Church St, Parramatta NSW'),

 -- Mon (breakfast/lunch/dinner)
 (26, 11, 1,'2025-09-22 09:10:00','takeaway','paid',     17.80, 1.78, 0.00, 19.58, NULL),
 (27, 12, 2,'2025-09-22 12:05:00','dine in','paid',      27.50, 2.75, 0.00, 30.25, NULL),
 (28, 13, 3,'2025-09-22 19:20:00','delivery','paid',     41.50, 4.15, 0.00, 45.65, '55 George St, Parramatta NSW'),

 -- Tue (breakfast/evening)
 (29, 14, 1,'2025-09-23 08:30:00','dine in','paid',      14.80, 1.48, 0.00, 16.28, NULL),
 (30, 15, 2,'2025-09-23 20:10:00','takeaway','paid',     30.00, 3.00, 0.00, 33.00, NULL);
```

## Entity 9: ORDER_DETAIL

```
-- ========================
-- ORDER DETAIL (10% tax lines; line totals are tax-inclusive)
-- ========================
INSERT INTO ORDER_DETAIL
 (Order_ID, Line_No, Product_ID, Unit_Price_At_Order, Quantity, Line_Discount, Tax_Rate_At_Order, Line_Total) VALUES
 -- 16: 2x Cappuccino + Almond Croissant
 (16,1, 2,4.50,2,0.00,10.00,  9.90),
 (16,2,15,5.50,1,0.00,10.00,  6.05),

 -- 17: Beef Burger + Flat White
 (17,1, 8,18.00,1,0.00,10.00,19.80),
 (17,2, 1, 4.50,1,0.00,10.00, 4.95),

 -- 18: 2x Beef Burger + Iced Latte + Brownie
 (18,1, 8,18.00,2,0.00,10.00,39.60),
 (18,2, 3, 5.50,1,0.00,10.00, 6.05),
 (18,3,11, 6.50,1,0.00,10.00, 7.15),

  -- 19: Long Black + Almond Croissant + Avo Toast
  (19,1, 5, 3.80,1,0.00,10.00, 4.18),
  (19,2,15, 5.50,1,0.00,10.00, 6.05),
  (19,3, 6,14.00,1,0.00,10.00,15.40),

  -- 20: Chicken Salad + Flat White + Cheesecake
  (20,1,10,16.50,1,0.00,10.00,18.15),
  (20,2, 1, 4.50,1,0.00,10.00, 4.95),
  (20,3,12, 7.00,1,0.00,10.00, 7.70),

  -- 21: 2x Beef Burger + 2x Lemon Tart with $1 discount
  (21,1, 8,18.00,2,0.00,10.00,39.60),
  (21,2,13, 6.00,2,1.00,10.00,12.10),

  -- 22: Cappuccino + Matcha + Almond Croissant
  (22,1, 2,4.50,1,0.00,10.00,4.95),
  (22,2, 4,5.00,1,0.00,10.00,5.50),
  (22,3,15,5.50,1,0.00,10.00,6.05),

  -- 23: Big Breakfast + Long Black + Cheesecake
  (23,1, 7,19.00,1,0.00,10.00,20.90),
  (23,2, 5, 3.80,1,0.00,10.00, 4.18),
  (23,3,12, 7.00,1,0.00,10.00, 7.70),

  -- 24: Veggie Wrap + Flat White
  (24,1,10,12.50,1,0.00,10.00,13.75),
  (24,2, 1, 4.50,1,0.00,10.00, 4.95),

  -- 25: 2x Chicken Salad + Brownie + Iced Latte
  (25,1,10,16.50,2,0.00,10.00,36.30),
  (25,2,11, 6.50,1,0.00,10.00, 7.15),
  (25,3, 3, 5.50,1,0.00,10.00, 6.05),

  -- 26: Avo Toast + Long Black
  (26,1, 6,14.00,1,0.00,10.00,15.40),
  (26,2, 5, 3.80,1,0.00,10.00, 4.18),
```

```
417      -- 27: Chicken Salad + Cappuccino + Brownie
418      (27,1,10,16.50,1,0.00,10.00,18.15),
419      (27,2, 2, 4.50,1,0.00,10.00, 4.95),
420      (27,3,11, 6.50,1,0.00,10.00, 7.15),
421
422      -- 28: Beef Burger + Veggie Wrap + Lemon Tart + Matcha
423      (28,1, 8,18.00,1,0.00,10.00,19.80),
424      (28,2,10,12.50,1,0.00,10.00,13.75),
425      (28,3,13, 6.00,1,0.00,10.00, 6.60),
426      (28,4, 4, 5.00,1,0.00,10.00, 5.50),
427
428      -- 29: Flat White + Chai Latte + Almond Croissant
429      (29,1, 1, 4.50,1,0.00,10.00, 4.95),
430      (29,2, 6, 4.80,1,0.00,10.00, 5.28),
431      (29,3,15, 5.50,1,0.00,10.00, 6.05),
432
433      -- 30: Beef Burger + Iced Latte + Brownie
434      (30,1, 8,18.00,1,0.00,10.00,19.80),
435      (30,2, 3, 5.50,1,0.00,10.00, 6.05),
436      (30,3,11, 6.50,1,0.00,10.00, 7.15);
```

## Entity 10: PAYMENT

```
438      -- ========================
439      -- PAYMENTS
440      -- ========================
441 •    INSERT INTO PAYMENT (Order_ID, Payment_Datetime, Amount, Payment_Method, Payment_Status) VALUES
442       (16,'2025-09-19 08:20:00',15.95,'card','approved'),
443       (17,'2025-09-19 12:15:00',24.75,'card','approved'),
444       (18,'2025-09-19 19:10:00',52.80,'online','approved'),
445       (19,'2025-09-20 09:05:00',25.63,'card','approved'),
446       (20,'2025-09-20 12:45:00',30.80,'card','approved'),
447       (21,'2025-09-20 19:40:00',51.70,'online','approved'),
448       (22,'2025-09-21 08:15:00',16.50,'card','approved'),
449       (23,'2025-09-21 13:10:00',32.78,'card','approved'),
450       (24,'2025-09-21 15:30:00',18.70,'card','approved'),
451       (25,'2025-09-21 18:50:00',49.50,'online','approved'),
452       (26,'2025-09-22 09:10:00',19.58,'card','approved'),
453       (27,'2025-09-22 12:05:00',30.25,'card','approved'),
454       (28,'2025-09-22 19:20:00',45.65,'online','approved'),
455       (29,'2025-09-23 08:30:00',16.28,'card','approved'),
456       (30,'2025-09-23 20:10:00',33.00,'card','approved');
457
```

## Entity 11: ROSTER

```
459     -- ========================
460     -- ROSTER (same outlet rule; some staff do 2 shifts/day)
461     -- ========================
462 •   INSERT INTO ROSTER
463       (Shift_ID, Staff_ID, Outlet_ID,
464        Scheduled_Start_Time, Scheduled_End_Time,
465        Actual_Start_Time,   Actual_End_Time,
466        Role_On_Shift)
467     VALUES
468       -- 2025-09-15 • OUTLET 1 • morning (Shift 1)
469       (1, 1, 1, '2025-09-15 08:00:00','2025-09-15 14:00:00','2025-09-15 08:05:00','2025-09-15 13:50:00','Manager'),
470       (1, 2, 1, '2025-09-15 08:00:00','2025-09-15 14:00:00','2025-09-15 08:03:00','2025-09-15 13:45:00','Chef'),
471       (1, 3, 1, '2025-09-15 08:00:00','2025-09-15 14:00:00','2025-09-15 08:00:00','2025-09-15 13:40:00','Server'),
472       (1, 4, 1, '2025-09-15 08:00:00','2025-09-15 14:00:00','2025-09-15 08:10:00','2025-09-15 13:55:00','Barista'),
473
474       -- 2025-09-15 • OUTLET 1 • evening (Shift 2)
475       (2, 1, 1, '2025-09-15 14:00:00','2025-09-15 22:00:00','2025-09-15 14:05:00','2025-09-15 21:50:00','Manager'),
476       (2, 2, 1, '2025-09-15 14:00:00','2025-09-15 22:00:00','2025-09-15 14:10:00','2025-09-15 21:45:00','Chef'),
477       (2, 3, 1, '2025-09-15 14:00:00','2025-09-15 22:00:00','2025-09-15 14:00:00','2025-09-15 21:35:00','Server'),
478       (2, 4, 1, '2025-09-15 14:00:00','2025-09-15 22:00:00','2025-09-15 14:00:00','2025-09-15 21:40:00','Barista'),
479
480       -- 2025-09-16 • OUTLET 2 • morning (Shift 9)
481       (9,  5, 2, '2025-09-16 08:00:00','2025-09-16 14:00:00','2025-09-16 08:00:00','2025-09-16 14:00:00','Manager'),
482       (9,  6, 2, '2025-09-16 08:00:00','2025-09-16 14:00:00','2025-09-16 08:02:00','2025-09-16 13:50:00','Chef'),
483       (9,  7, 2, '2025-09-16 08:00:00','2025-09-16 14:00:00','2025-09-16 08:05:00','2025-09-16 13:45:00','Server'),
484       (9,  8, 2, '2025-09-16 08:00:00','2025-09-16 14:00:00','2025-09-16 08:00:00','2025-09-16 13:42:00','Barista'),
485
486       -- 2025-09-16 • OUTLET 2 • evening (Shift 10)
487       (10, 5, 2, '2025-09-16 14:00:00','2025-09-16 22:00:00','2025-09-16 14:00:00','2025-09-16 22:00:00','Manager'),
488       (10, 6, 2, '2025-09-16 14:00:00','2025-09-16 22:00:00','2025-09-16 14:07:00','2025-09-16 21:43:00','Chef'),
489       (10, 7, 2, '2025-09-16 14:00:00','2025-09-16 22:00:00','2025-09-16 14:10:00','2025-09-16 21:45:00','Server'),
490       (10, 8, 2, '2025-09-16 14:00:00','2025-09-16 22:00:00','2025-09-16 14:00:00','2025-09-16 21:50:00','Barista'),
491
492       -- 2025-09-17 • OUTLET 3 • morning (Shift 17)
493       (17, 9,  3, '2025-09-17 08:00:00','2025-09-17 14:00:00','2025-09-17 08:00:00','2025-09-17 13:40:00','Manager'),
494       (17, 10, 3, '2025-09-17 08:00:00','2025-09-17 14:00:00','2025-09-17 08:09:00','2025-09-17 13:50:00','Chef'),
495       (17, 11, 3, '2025-09-17 08:00:00','2025-09-17 14:00:00','2025-09-17 08:05:00','2025-09-17 13:35:00','Server'),
496       (17, 12, 3, '2025-09-17 08:00:00','2025-09-17 14:00:00','2025-09-17 08:00:00','2025-09-17 13:54:00','Barista'),
497
498       -- 2025-09-17 • OUTLET 3 • evening (Shift 18)
499       (18, 9,  3, '2025-09-17 14:00:00','2025-09-17 22:00:00','2025-09-17 14:05:00','2025-09-17 21:45:00','Manager'),
500       (18, 10, 3, '2025-09-17 14:00:00','2025-09-17 22:00:00','2025-09-17 14:00:00','2025-09-17 21:42:00','Chef'),
501       (18, 11, 3, '2025-09-17 14:00:00','2025-09-17 22:00:00','2025-09-17 14:00:00','2025-09-17 21:35:00','Server'),
502       (18, 12, 3, '2025-09-17 14:00:00','2025-09-17 22:00:00','2025-09-17 14:10:00','2025-09-17 21:58:00','Barista');
503
```

# Part B – Business Analytics Report

## 2.1 Business concern 1

**Name and Description: Demand forecasting and capacity planning**

Traffic varies by outlet, day of the week, hour, and order type (dine-in/takeaway/delivery). Without strong forecasting, Urban Eats Café Chain might have concerns about either under- or overstaffing and missed sales.

**SQL Query:**

```
506     -- 1) Demand forecasting & capacity planning
507     -- Peak demand by outlet x day-of-week x hour (last 8 weeks)
508 •   SELECT
509       Outlet_ID,
510       DATE_FORMAT(Order_Datetime, '%a') AS dow,
511       HOUR(Order_Datetime) AS hr,
512       COUNT(*) AS orders,
513       ROUND(SUM(Grand_Total),2) AS revenue
514     FROM CUSTOMER_ORDER
515     WHERE Order_Status = 'paid'
516       AND Order_Datetime >= DATE_SUB(CURDATE(), INTERVAL 56 DAY)
517     GROUP BY Outlet_ID, dow, hr
518     ORDER BY orders DESC, revenue DESC;
```

**SQL Result:**

| Outlet_ID | dow | hr | orders | revenue |
|---|---|---|---|---|
| 1 | Sun | 13 | 1 | 32.78 |
| 1 | Fri | 12 | 1 | 24.75 |
| 1 | Mon | 9 | 1 | 19.58 |
| 1 | Sun | 8 | 1 | 16.50 |
| 1 | Tue | 8 | 1 | 16.28 |
| 1 | Fri | 8 | 1 | 15.95 |
| 2 | Fri | 19 | 1 | 52.80 |
| 2 | Sat | 19 | 1 | 51.70 |
| 2 | Tue | 20 | 1 | 33.00 |
| 2 | Mon | 12 | 1 | 30.25 |
| 2 | Sun | 15 | 1 | 18.70 |
| 3 | Sun | 18 | 1 | 49.50 |
| 3 | Mon | 19 | 1 | 45.65 |
| 3 | Sat | 12 | 1 | 30.80 |
| 3 | Sat | 9 | 1 | 25.63 |

**Justification and Insights:**

The decision support system is important in identifying peak hour blocks for each outlet, ensuring that SHIFT and ROSTER coverage is effectively aligned. This information is vital for informing production planning, including factors such as batch brew and

pastry bake times. To optimize operations, it is recommended to establish baseline staffing guides tailored to each outlet. For example, during weekdays from 12 to 2 PM, Outlet 1 should be staffed with one Manager, one Chef, two Servers, and one Barista. It is also important to review these staffing guides monthly and make adjustments as needed to account for seasonality and specific events.

## 2.2 Business concern 2

**Name and Description: Reservation reliability and table utilisation**

Managing reservation statuses like requested, confirmed, cancelled, and no-shows is challenging. No-shows waste capacity, and overbooking can harm customer experience. The lack of a TABLE entity in the system limits utilisation analytics. By effectively tracking no-show and cancellation rates by outlet, Urban Eats Café Chain can refine confirmation procedures and improve its overbooking policies to enhance efficiency.

**SQL Query:**

```
521    -- 2) Reservation reliability & table utilisation
522    -- No-show & cancellation rates by outlet (last 90 days)
523    SELECT
524      Outlet_ID,
525      COUNT(*) AS reservations,
526      SUM(Reservation_Status = 'no_show')    AS no_shows,
527      SUM(Reservation_Status = 'cancelled')  AS cancellations,
528      ROUND(SUM(Reservation_Status = 'no_show')
529            / NULLIF(SUM(Reservation_Status IN ('requested','confirmed')),0), 3) AS no_show_rate,
530      ROUND(SUM(Reservation_Status = 'cancelled') / NULLIF(COUNT(*),0), 3)       AS cancel_rate
531    FROM RESERVATION
532    WHERE Reservation_Datetime >= DATE_SUB(CURDATE(), INTERVAL 90 DAY)
533    GROUP BY Outlet_ID
534    ORDER BY no_show_rate DESC, cancel_rate DESC;
```

**SQL Result:**

| Outlet_ID | reservations | no_shows | cancellations | no_show_rate | cancel_rate |
|---|---|---|---|---|---|
| 2 | 4 | 1 | 2 | 1.000 | 0.500 |
| 1 | 4 | 0 | 0 | 0.000 | 0.000 |
| 3 | 4 | 0 | 0 | 0.000 | 0.000 |

**Justification and Insights**

To enhance the decision support capabilities, Urban Eats Café Chain aims to quantify the reliability of each outlet. This will enable us to determine the most effective tactics for sending confirmations and reminders, as well as establishing an appropriate overbooking buffer. For outlets experiencing a no-show rate exceeding 10%, it is advisable to implement SMS reminders and require a deposit for peak time slots. Additionally, an enhancement to the system could involve adding a TABLES entity along with seat capacity, which would allow us to accurately compute the true percentage of utilization for each time block.

## 2.3 Business concern 3

**Name and Description: Channel (order type) profitability**

Channel profitability varies across dine-in, takeaway, and especially delivery services, each with distinct costs and margins. While total revenues are recorded, delivery fees, commissions, and rider costs often remain untracked, complicating the assessment of true profitability. Revenue and refunds differ by channel. Even without COGS/fees, Urban Eats Café Chain can monitor net collected, such as approved payments minus refunds and refund rate by channel.

**SQL Query**

```
536     -- 3) Channel profitability
537     -- Net revenue & refund rate by channel
538     WITH pay AS (
539       SELECT
540         co.Order_ID,
541         co.Order_Type,
542         SUM(CASE WHEN p.Payment_Status = 'approved' THEN p.Amount ELSE 0 END) AS approved_amt,
543         SUM(CASE WHEN p.Payment_Status = 'refunded' THEN p.Amount ELSE 0 END) AS refunded_amt
544       FROM CUSTOMER_ORDER co
545       LEFT JOIN PAYMENT p ON p.Order_ID = co.Order_ID
546       GROUP BY co.Order_ID, co.Order_Type
547     )
```

```
548     SELECT
549       Order_Type,
550       COUNT(*) AS orders,
551       ROUND(SUM(approved_amt),2) AS gross_collected,
552       ROUND(SUM(refunded_amt),2) AS refunds,
553       ROUND(SUM(approved_amt - refunded_amt),2) AS net_revenue,
554       ROUND(SUM(refunded_amt)/NULLIF(SUM(approved_amt),0),3) AS refund_rate
555     FROM pay
556     GROUP BY Order_Type
557     ORDER BY net_revenue ASC, refund_rate DESC;
```

**SQL Result**

| Order_Type | orders | gross_collected | refunds | net_revenue | refund_rate |
|---|---|---|---|---|---|
| takeaway | 5 | 126.83 | 0.00 | 126.83 | 0.000 |
| dine in | 6 | 137.39 | 0.00 | 137.39 | 0.000 |
| delivery | 4 | 199.65 | 0.00 | 199.65 | 0.000 |

**Justification and Insights:**

Using decision support tools can provide valuable insights into various channels, particularly those that exhibit higher refund rates or lower net average order value. This information is necessary as it can help guide pricing strategies, promotional activities, and areas of service that may need more focus. In terms of recommendations, it's important to track delivery commissions and fees by adding extra columns to your data. This will provide a clearer picture of the true margin. If the net performance is weak, consider adjusting the menu prices or creating delivery-only bundles to optimize profitability.

## 2.4 Business concern 4

**Name and Description: Menu engineering and optimisation**

Menu engineering and optimization are ongoing processes aimed at enhancing our beverages, meals, and desserts. Urban Eats focuses on identifying high and low performers by sales and adjusting pricing strategies accordingly. However, the lack of Cost of Goods Sold (COGS) data prevents us from understanding proper item margins.

### SQL Query

```
560     -- 4) Menu engineering & optimisation
561     -- Product performance (last 60 days): low performers first
562 •   SELECT
563       pc.Category_Name,
564       p.Product_ID,
565       p.Product_Name,
566       SUM(od.Quantity) AS qty_sold,
567       ROUND(SUM(od.Line_Total),2) AS revenue
568     FROM ORDER_DETAIL od
569     JOIN CUSTOMER_ORDER co ON co.Order_ID = od.Order_ID
570     JOIN PRODUCT p         ON p.Product_ID = od.Product_ID
571     JOIN PRODUCT_CATEGORY pc ON pc.Category_ID = p.Category_ID
572     WHERE co.Order_Status = 'paid'
573       AND co.Order_Datetime >= DATE_SUB(CURDATE(), INTERVAL 60 DAY)

574     GROUP BY pc.Category_Name, p.Product_ID, p.Product_Name
575     ORDER BY qty_sold ASC, revenue ASC;
```

### SQL Result

| Category_Name | Product_ID | Product_Name | qty_sold | revenue |
|---|---|---|---|---|
| Meals | 7 | Big Breakfast | 1 | 20.90 |
| Beverages | 4 | Matcha Latte | 2 | 11.00 |
| Desserts | 12 | Cheesecake Slice | 2 | 15.40 |
| Beverages | 5 | Long Black | 3 | 12.54 |
| Beverages | 3 | Iced Latte | 3 | 18.15 |
| Desserts | 13 | Lemon Tart | 3 | 18.70 |
| Meals | 6 | Smashed Avo Toast | 3 | 36.08 |
| Beverages | 1 | Flat White | 4 | 19.80 |
| Beverages | 2 | Cappuccino | 4 | 19.80 |
| Desserts | 15 | Almond Croissant | 4 | 24.20 |
| Desserts | 11 | Chocolate Brownie | 4 | 28.60 |
| Meals | 10 | Veggie Wrap | 6 | 100.10 |
| Meals | 8 | Beef Burger | 7 | 138.60 |

**Justification and Insights:** To enhance the menu at Urban Eats Coffee Chain, it's essential to identify star items that drive high sales and revenue while removing low-performing dishes. By promoting successful items and adjusting their pricing and recipes, Urban Eats can increase profitability and customer satisfaction. The focus should be on determining which items should be featured, re-priced, or retired, while also guiding promotional planning and setting preparation priorities. To achieve a better balance among beverages, meals, and desserts, it is advisable to include the cost of

goods sold (COGS) for each product. This will allow for tracking item-level margins and enable more precise segmentation based on profitability rather than just revenue.

## 2.5 Business concern 5

**Name and Description: Staff scheduling and performance**

Managing the discrepancy between actual hours worked and scheduled hours can be quite challenging, especially when dealing with double shifts within a single day and ensuring that employees adhere to their designated break times. While I calculate the total Hours_Worked, it is crucial to implement a monitoring system to track instances of early or late clock-ins.

**SQL Query**

```
578   -- 5) Staff scheduling & performance
579   SELECT
580     r.Outlet_ID,
581     r.Staff_ID,
582     DATE(COALESCE(r.Actual_Start_Time, r.Scheduled_Start_Time)) AS work_date,
583     ROUND(TIMESTAMPDIFF(MINUTE, r.Scheduled_Start_Time, r.Scheduled_End_Time)/60, 2) AS scheduled_hours,
584     ROUND(COALESCE(r.Hours_Worked,
585                    TIMESTAMPDIFF(MINUTE, r.Scheduled_Start_Time, r.Scheduled_End_Time)/60), 2) AS actual_hours,
586     ROUND(
587       COALESCE(r.Hours_Worked,
588                TIMESTAMPDIFF(MINUTE, r.Scheduled_Start_Time, r.Scheduled_End_Time)/60)
589       - (TIMESTAMPDIFF(MINUTE, r.Scheduled_Start_Time, r.Scheduled_End_Time)/60), 2
590     ) AS variance_hours,                                  -- + = over, - = under

591     TIMESTAMPDIFF(MINUTE, r.Scheduled_Start_Time, r.Actual_Start_Time) AS start_delta_min, -- + = late, - = early
592     TIMESTAMPDIFF(MINUTE, r.Scheduled_End_Time, r.Actual_End_Time)     AS end_delta_min    -- + = stayed late, - = left early
593   FROM ROSTER r
594   ORDER BY work_date, r.Outlet_ID, r.Staff_ID;

596   SELECT
597     r.Outlet_ID,
598     r.Staff_ID,
599     DATE(COALESCE(r.Actual_Start_Time, r.Scheduled_Start_Time)) AS work_date,
600     COUNT(*) AS shifts_that_day,
601     ROUND(SUM(COALESCE(r.Hours_Worked,
602                        TIMESTAMPDIFF(MINUTE, r.Scheduled_Start_Time, r.Scheduled_End_Time)/60)), 2) AS hours_in_day
603   FROM ROSTER r
604   GROUP BY r.Outlet_ID, r.Staff_ID, work_date
605   HAVING COUNT(*) > 1
606   ORDER BY hours_in_day DESC;
607
```

```sql
SELECT
  s.Outlet_ID,
  s.Staff_ID,
  s.Staff_First_Name, s.Staff_Last_Name,
  COUNT(*) AS shifts,
  SUM(r.Actual_Start_Time IS NOT NULL AND TIMESTAMPDIFF(MINUTE, r.Scheduled_Start_Time, r.Actual_Start_Time) > 0)
  AS late_starts,
  ROUND(100.0 * SUM(r.Actual_Start_Time IS NULL OR TIMESTAMPDIFF
  (MINUTE, r.Scheduled_Start_Time, r.Actual_Start_Time) <= 0) / COUNT(*), 1) AS on_time_start_pct
FROM ROSTER r
JOIN STAFF s ON s.Staff_ID = r.Staff_ID
GROUP BY s.Outlet_ID, s.Staff_ID, s.Staff_First_Name, s.Staff_Last_Name
ORDER BY on_time_start_pct ASC, shifts DESC
LIMIT 15;
```

**SQL Result**

| Outlet_ID | Staff_ID | Staff_First_Name | Staff_Last_Name | shifts | late_starts | on_time_start_pct |
|---|---|---|---|---|---|---|
| 1 | 1 | Ava | Nguyen | 2 | 2 | 0.0 |
| 1 | 2 | Liam | Patel | 2 | 2 | 0.0 |
| 2 | 6 | Ethan | Taylor | 2 | 2 | 0.0 |
| 2 | 7 | Grace | Wilson | 2 | 2 | 0.0 |
| 1 | 4 | Noah | Price | 2 | 1 | 50.0 |
| 3 | 9 | Emily | Clark | 2 | 1 | 50.0 |
| 3 | 10 | Leo | Harris | 2 | 1 | 50.0 |
| 3 | 11 | Sophie | Cooper | 2 | 1 | 50.0 |
| 3 | 12 | Lucas | King | 2 | 1 | 50.0 |
| 1 | 3 | Mia | Zhou | 2 | 0 | 100.0 |
| 2 | 5 | Isla | Brown | 2 | 0 | 100.0 |
| 2 | 8 | Jack | Martin | 2 | 0 | 100.0 |

**Justification and Insights**

Labor is the most considerable controllable cost for Urban Eats and a significant issue in service quality. Variances between scheduled and actual hours lead to inflated overtime, increased idle time, and a diminished guest experience. Issues such as late clock-ins, early departures, and double shifts are associated with slower service, higher refund rates, and lower staff morale.

Since staff cannot rotate between different outlets, any misalignment at one location is amplified. To effectively monitor labor performance, focus on the following key insights:

- Punctuality metrics

- Actual versus scheduled hours variance by role and time of day

- Sales per labor hour, broken down by outlet and time of day.

## 2.6 Conclusion

This assignment involved designing and implementing a functioning database for Urban Eats, from initial concept development to data management. The individual first created a Chen-notation Entity-Relationship Diagram (ERD) that captured elements such as outlets, staff, shifts, customers, reservations, products, orders, and payments, along with two associative entities, which are ROSTER (linking staff to shifts) and ORDER_DETAIL (linking orders to products), tracking hours worked and line-item pricing snapshots. Using these keys, cardinalities, and business rules, a MySQL schema is developed that incorporates primary keys, foreign keys, checks, and enumerations. Sample data was then populated into the database to validate its integrity and generate meaningful insights related to orders, payments, reservations, and rosters. With this dataset, the individual analyzed five key business concerns: (1) Demand forecasting and capacity planning, (2) Reservation reliability and table utilisation, (3) Channel (order type) profitability, (4) Menu engineering and optimisation, and (5) Staff scheduling and performance. For each concern, the data-driven justifications and proposed targeted recommendations are provided. The outcome is a reliable foundation for daily operations and future analytical endeavors.